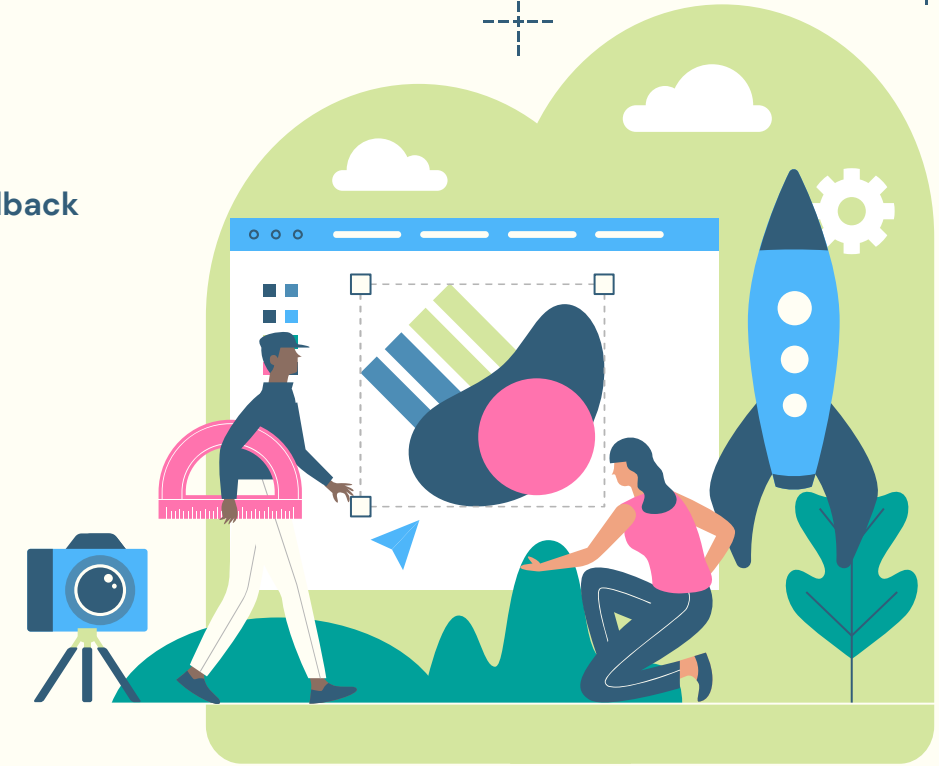
Student Grade Evaluation & Personalized Feedback System

Python Programming Mini Project

B.Tech Computer Science & Engineering – 3rd Semester



Marks & Regret

Student Grade Evaluation & Personalized Feedback System

Python Programming Mini Project

B.Tech Computer Science & Engineering - 3rd Semester

Team Members

- **EKANSH KUMAR — Project Overview & Introduction Coordinator**
- **AKSHATH MAURYA** - Project Lead & Requirements Analyst
- **DIVYANSHU NAHATA** - Solution Architect & Feature Designer
- **DANIEL CHRISTOPHER** - Python Developer & Module Designer
- **J S RAHUL**- System Implementation & Workflow Engineer
- **ARDRA RAJESH** - Quality Assurance & Testing Lead
- **GURRAMPATI HARSHA VARDHAN RATHOD** - Results, Evaluation & Documentation Lead

Project Lead: AKSHATH MAURYA

From Marks to Meaning

Students often receive a score without immediately knowing their grade, performance level, personalized feedback, or how many more marks they need to reach the next grade.
Manual calculation of these details can be repetitive and time-consuming.

Our project bridges this gap by automatically converting marks into meaningful performance insights.



Limited Feedback, Hidden Rules

Manual Evaluation:
Manual calculations can be repetitive and may lead to calculation or data-entry errors. Students receive marks, but understanding their performance and next target may require additional effort.

Basic Automated Evaluation:
Automated systems can calculate results quickly, but simple grading systems often stop at showing the grade and percentage without explaining what the student should aim for next.

Combine automated grading with immediate feedback and a clear next-grade target.

Automated Grading

The system automates student performance evaluation using rule-based conditions. Based on the entered marks, it determines:

Grade • Result • Grade Point • Performance Status

It also generates marks-based feedback and calculates the marks required to reach the next grade.
Input validation and exception handling handle empty names, non-numeric marks, and marks outside the 0–100 range.

Why This Problem

Students need quick and meaningful feedback to understand their performance and improve. This problem is simple enough to implement using Python while allowing us to apply important concepts such as conditional logic, functions, input validation, and exception handling. This project provides a foundation for future enhancements such as performance tracking, analytics, and personalized study recommendations.



Modular Solution

Our solution is a modular Python application that accepts a student's name and marks, validates the input, and automatically evaluates academic performance. The system determines the grade, pass/fail result, grade point, performance status, personalized feedback, and marks required for the next grade. Dividing the program into separate functions makes it simple, organized, and easy to maintain.

Objectives & Features

Instant Grade Check

Enter marks → Get your grade instantly

No More Manual Calculations

Automatically determines: Grade • Result • Grade Point

Reality Check

Get a fun, marks-based performance status:
TOPPER MODE
ALMOST TOPPER
NOT BAD
YOU CAN DO BETTER
BARELY SURVIVING
ACADEMIC EMERGENCY

Marks-Based Feedback

The system provides different feedback messages based on the student's marks.

Know Your Next Target

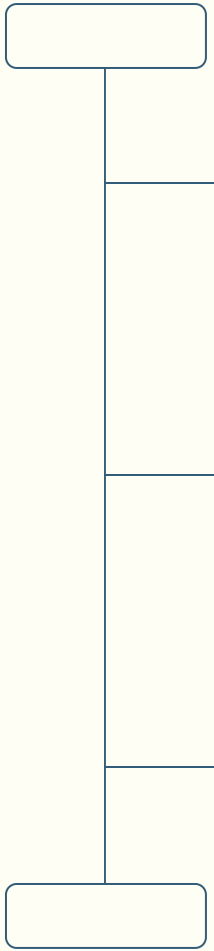
Find out exactly how many marks you need to reach the next grade.

Smart Input Validation

Checks empty names, invalid/non-numeric input, and marks outside 0–100, with clear error messages.

Don't just know your marks. Know what they mean.

System Workflow



Collect & Validate

The system collects the student's name and marks through user input and validates the entered information.

Process & Evaluate

The program processes the marks using conditional logic to determine the grade, result, grade point, and performance status.

Feedback & Report

The system generates personalized feedback and the next-grade target, then displays all the results in a structured report.

Python Functions & Modular Design

Modular Design

The program is divided into separate functions, with each function responsible for one specific task. This makes the code easier to understand, test, debug, and maintain.

Function Flow

User Input → Validation → Grade Calculation → Feedback → Next Target → Final Report

Modular Architecture

main()
↓
Input & Validation
↓
calculate_grade_and_result()
↓
display_report()
├── **get_feedback()**
└── **marks_needed_for_next_grade()**

Why Modular?

Separate Tasks → Cleaner Code → Easier Testing → Easier Maintenance

Modular Functions

calculate_grade_and_result()

Evaluates the marks and returns:

Grade • Result • Grade Point • Performance Status

get_feedback()

Generates personalized feedback based on the student's marks.

marks_needed_for_next_grade()

Calculates how many additional marks are required to reach the next grade.

display_report()

Combines the calculated information and displays the final student report.

main()

Controls the overall program flow, handles user input, validation, and function calls.

Single Responsibility:

Each function performs one major task, making the program easier to understand, test, debug, and modify.

Proof by Screenshots

The screenshots demonstrate the system working under different input conditions.

01 — Valid Input

Student name and marks are entered successfully.

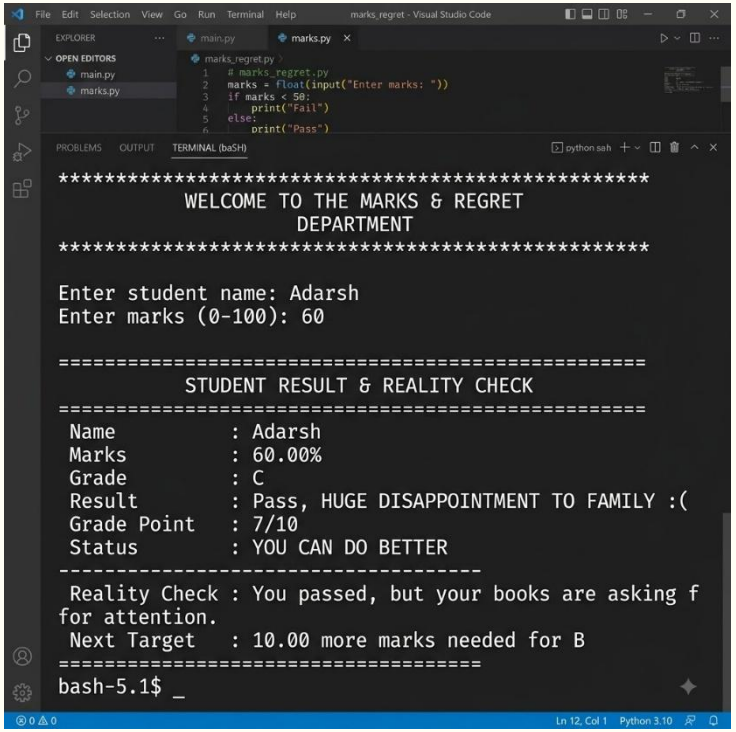
02 — Generated Result

The system displays the grade, result, grade point, performance status, feedback, and next-grade target.

03 — Invalid Input

The system detects invalid marks, marks outside the 0 -100 range, or invalid input and displays an appropriate error message.

The screenshots provide practical evidence that the implemented functions work as expected.



Test Results

The system was manually tested using valid, boundary, and invalid inputs to verify the implemented grading rules and error-handling logic.

Result: The implemented rules and validation logic produced the expected outputs during testing.

Grade Rule Testing

A+ to F conditions verified
Boundary values such as 50, 60, 70, 80, and 90 were checked to confirm correct grade transitions.

Input Validation

Invalid inputs handled
Tested with empty names, non-numeric marks, negative values, and marks above 100.

Marks-based outputs verified

Feedback & Target Generation
Feedback messages and next-grade targets were checked across different mark ranges.

Fast, Correct, with Growth

Current Strengths

The system automates grade calculation and reduces repetitive manual effort by applying clearly defined grading rules. It generates marks-based feedback, calculates the marks required for the next grade, validates user input, and handles common input errors. Its modular function-based design also makes the program easier to understand, test, debug, and maintain.

Limitations & Future Growth

The current system processes one student at a time, uses rule-based feedback, does not permanently store data, and displays results through the console. Future versions can extend the system with multiple-student processing, CSV storage, performance analytics, graphical reports, and AI-based personalized recommendations.

Automate today.

Get better tomorrow.

Results & Access

Results

The system successfully automates the evaluation of student marks and generates the grade, pass/fail result, grade point, performance status, marks-based feedback, and the marks required to reach the next grade. It also validates user input, preventing invalid marks and handling incorrect input effectively.

Future Scope

The system can be further enhanced to support multiple students, data storage, performance analytics, graphical reports, and AI-based recommendations. These improvements can make the system more useful for tracking student performance and providing more advanced academic insights.

Project Access

GitHub Repository: https://ardra07-rajesh.github.io/student_grade_evaluation.py/

Website: marks-and-regret.html

Closing Thoughts

- Automates grade computation and marks-based feedback using clear rule-based logic.
- Uses a modular Python design with readable and maintainable functions.
- Applies important Python concepts including conditional statements, functions, input validation, and exception handling.
- Provides immediate results, performance status, feedback, and next-grade targets.
- Testing confirms the system's behavior across valid, boundary, and invalid inputs.
- Future enhancements can include multi-student processing, data storage, analytics, and AI-based recommendations.

Thanks!

From Marks to Meaning.
Turning student scores into actionable feedback.

Do you have any questions?

